

lab5_part2

```
1  function lab5_part2()
2  % =====
3  % Lab 5 - Part B
4  % Open Jackson Queueing Network
5  %
6  % File name:
7  %   lab5_part2.m
8  %
9  % Run in Octave with:
10 %   lab5_part2
11 %
12 % Implements:
13 %   B.2 intensities(lambda1, lambda2, mu1, ..., mu5)
14 %   B.3 mean_clients(lambda1, lambda2, mu1, ..., mu5)
15 %   B.4 numerical evaluation for the given parameters
16 %   B.5 bottleneck queue and maximum lambda1 for ergodicity
17 %   B.6 end-to-end mean delay plot versus lambda1
18 %
19 % Network routing used:
20 %   Q1 -> Q2      with probability 1/2
21 %   Q1 -> Q3      with probability 1/4
22 %   Q1 -> exit    with probability 1/4
23 %
24 %   Q2 -> Q3      with probability 1/3
25 %   Q2 -> Q4      with probability 2/3
26 %
27 %   Q3 -> Q5      with probability 1
28 %
29 %   Q4 -> Q5      with probability 3/5
30 %   Q4 -> exit    with probability 2/5
31 %
32 %   Q5 -> exit    with probability 1
33 %
34 % Effective arrival rates:
35 %   Gamma1 = lambda1
36 %   Gamma2 = lambda2 + (1/2)lambda1
37 %   Gamma3 = (5/12)lambda1 + (1/3)lambda2
38 %   Gamma4 = (1/3)lambda1 + (2/3)lambda2
39 %   Gamma5 = (37/60)lambda1 + (11/15)lambda2
40 %
41 % All rates are in clients/sec, so delays are in seconds.
42 % =====
43
44 clc;
45 close all;
46 format long g;
47
48 % -----
49 % Given numerical parameters from the statement
50 % -----
```

```

51
52 lambda1 = 10;
53 lambda2 = 6;
54
55 mu1 = 15;
56 mu2 = 20;
57 mu3 = 10;
58 mu4 = 14;
59 mu5 = 12;
60
61 mu = [mu1, mu2, mu3, mu4, mu5];
62
63 fprintf('\n===== \n');
64 fprintf('Lab 5 - Part B: Open Jackson Queueing Network\n');
65 fprintf('===== \n');
66
67 fprintf('\nInput parameters:\n');
68 fprintf(' lambda1 = %.10f clients/sec\n', lambda1);
69 fprintf(' lambda2 = %.10f clients/sec\n', lambda2);
70 fprintf(' mu1      = %.10f clients/sec\n', mu1);
71 fprintf(' mu2      = %.10f clients/sec\n', mu2);
72 fprintf(' mu3      = %.10f clients/sec\n', mu3);
73 fprintf(' mu4      = %.10f clients/sec\n', mu4);
74 fprintf(' mu5      = %.10f clients/sec\n', mu5);
75
76 % -----
77 % B.2 - Effective arrival rates and intensities
78 % -----
79
80 fprintf('\n===== \n');
81 fprintf('B.2 - Effective arrival rates and traffic intensities\n');
82 fprintf('===== \n');
83
84 gamma = effective_arrival_rates(lambda1, lambda2);
85
86 fprintf('\nEffective arrival rates Gamma_i:\n');
87 for i = 1:5
88     fprintf(' Gamma%d = %.10f clients/sec\n', i, gamma(i));
89 endfor
90
91 [rho, ergodic] = intensities(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5);
92
93 fprintf('\nErgodicity flag: %d\n', ergodic);
94 if ergodic == 1
95     fprintf('The network is ergodic because all rho_i are smaller than 1.\n');
96 else
97     fprintf('The network is not ergodic because at least one rho_i is >= 1.\n');
98 endif
99
100 % -----
101 % B.3 - Mean number of clients in each queue
102 % -----
103
104 fprintf('\n===== \n');

```

```

105 fprintf('B.3 - Mean number of clients in each queue\n');
106 fprintf('===== \n');
107
108 N = mean_clients(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5);
109
110 fprintf('\nMean clients E[N_i] in each queue:\n');
111 for i = 1:5
112     fprintf(' E[N%d] = %.10f\n', i, N(i));
113 endfor
114
115 N_total = sum(N);
116
117 fprintf('\nTotal mean number of clients in the network:\n');
118 fprintf(' E[N] = sum_i E[N_i] = %.10f\n', N_total);
119
120 % -----
121 % B.4 - End-to-end mean delay for the given parameters
122 % -----
123
124 fprintf('\n===== \n');
125 fprintf('B.4 - End-to-end mean delay for the given parameters\n');
126 fprintf('===== \n');
127
128 external_rate = lambda1 + lambda2;
129 T_end_to_end = end_to_end_delay(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5);
130
131 fprintf('\nTotal external arrival rate:\n');
132 fprintf(' gamma_external = lambda1 + lambda2 = %.10f clients/sec\n', external_rate);
133
134 fprintf('\nEnd-to-end mean delay by Little''s Law:\n');
135 fprintf(' E[T] = E[N] / gamma_external = %.10f seconds\n', T_end_to_end);
136
137 % -----
138 % B.5 - Bottleneck and maximum lambda1 for ergodicity
139 % -----
140
141 fprintf('\n===== \n');
142 fprintf('B.5 - Bottleneck queue and maximum lambda1\n');
143 fprintf('===== \n');
144
145 [max_rho, bottleneck_current] = max(rho);
146
147 fprintf('\nCurrent bottleneck queue, based on maximum rho_i:\n');
148 fprintf(' bottleneck = Q%d\n', bottleneck_current);
149 fprintf(' max rho = %.10f\n', max_rho);
150
151 [lambda1_max, limiting_queue, bounds] = lambda1_max_for_ergodicity(lambda2, mu1, mu2,
mu3, mu4, mu5);
152
153 fprintf('\nUpper bounds on lambda1 imposed by each queue:\n');
154 for i = 1:5
155     fprintf(' From Q%d: lambda1 < %.10f\n', i, bounds(i));
156 endfor
157

```

```

158 fprintf('\nMaximum lambda1 for ergodicity with lambda2 fixed at %.10f:\n', lambda2);
159 fprintf(' lambda1_max = %.10f clients/sec\n', lambda1_max);
160 fprintf(' limiting queue = Q%d\n', limiting_queue);
161
162 if lambda1 < lambda1_max
163     fprintf(' Current lambda1 = %.10f is below lambda1_max, so the current network is
ergodic.\n', lambda1);
164 else
165     fprintf(' Current lambda1 = %.10f is not below lambda1_max, so the current
network is not ergodic.\n', lambda1);
166 endif
167
168 % -----
169 % B.6 - Plot end-to-end delay versus lambda1
170 % -----
171
172 fprintf('\n===== \n');
173 fprintf('B.6 - End-to-end delay plot versus lambda1\n');
174 fprintf('===== \n');
175
176 lambda1_values = linspace(0.1 * lambda1_max, 0.99 * lambda1_max, 300);
177 delay_values = zeros(size(lambda1_values));
178
179 for k = 1:length(lambda1_values)
180     delay_values(k) = end_to_end_delay(lambda1_values(k), lambda2, ...
181                                     mu1, mu2, mu3, mu4, mu5);
182 endfor
183
184 fprintf('\nPlot range:\n');
185 fprintf(' lambda1_min = %.10f clients/sec\n', lambda1_values(1));
186 fprintf(' lambda1_max_plot = %.10f clients/sec\n', lambda1_values(end));
187 fprintf(' lambda1_max_ergodic = %.10f clients/sec\n', lambda1_max);
188
189 fprintf('\nDelay values at selected points:\n');
190
191 selected_factors = [0.10, 0.25, 0.50, 0.75, 0.90, 0.95, 0.99];
192 for k = 1:length(selected_factors)
193     l1_sel = selected_factors(k) * lambda1_max;
194     d_sel = end_to_end_delay(l1_sel, lambda2, mu1, mu2, mu3, mu4, mu5);
195
196     fprintf(' lambda1 = %.2f * lambda1_max = %.10f -> E[T] = %.10f sec\n', ...
197             selected_factors(k), l1_sel, d_sel);
198 endfor
199
200 figure(1);
201 clf;
202
203 plot(lambda1_values, delay_values, 'LineWidth', 1.5);
204 grid on;
205 xlabel('lambda_1 (clients/sec)');
206 ylabel('End-to-end mean delay E[T] (sec)');
207 title('B.6 - End-to-end mean delay versus lambda_1');
208
209 hold on;

```

```

210 y1 = ylim();
211 plot([lambda1, lambda1], y1, '--', 'LineWidth', 1.0);
212 plot([lambda1_max, lambda1_max], y1, ':', 'LineWidth', 1.0);
213 legend('E[T]', 'Given lambda_1 = 10', 'lambda_{1,max}', 'Location', 'northwest');
214 hold off;
215
216 try
217     print(gcf, 'lab5_part2_delay.png', '-dpng', '-r200');
218     fprintf('\nSaved figure as lab5_part2_delay.png\n');
219 catch
220     fprintf('\nCould not save figure automatically. Please export it manually if
needed.\n');
221 end_try_catch
222
223 % -----
224 % Final summary for easy copy into report
225 % -----
226
227 fprintf('\n=====\\n');
228 fprintf('FINAL SUMMARY - Lab 5, Part B\\n');
229 fprintf('=====\\n');
230
231 fprintf('\\nRouting equations:\\n');
232 fprintf('  Gamma1 = lambda1\\n');
233 fprintf('  Gamma2 = lambda2 + (1/2)lambda1\\n');
234 fprintf('  Gamma3 = (5/12)lambda1 + (1/3)lambda2\\n');
235 fprintf('  Gamma4 = (1/3)lambda1 + (2/3)lambda2\\n');
236 fprintf('  Gamma5 = (37/60)lambda1 + (11/15)lambda2\\n');
237
238 fprintf('\\nGiven parameters:\\n');
239 fprintf('  lambda1 = %.10f, lambda2 = %.10f\\n', lambda1, lambda2);
240 fprintf('  mu = [%.10f %.10f %.10f %.10f %.10f]\\n', mu1, mu2, mu3, mu4, mu5);
241
242 fprintf('\\nEffective arrival rates:\\n');
243 fprintf('  Gamma = [');
244 fprintf(' %.10f', gamma);
245 fprintf(' ]\\n');
246
247 fprintf('\\nTraffic intensities:\\n');
248 fprintf('  rho = [');
249 fprintf(' %.10f', rho);
250 fprintf(' ]\\n');
251 fprintf('  ergodic = %d\\n', ergodic);
252
253 fprintf('\\nMean clients:\\n');
254 fprintf('  E[N_i] = [');
255 fprintf(' %.10f', N);
256 fprintf(' ]\\n');
257 fprintf('  E[N] = %.10f\\n', N_total);
258
259 fprintf('\\nEnd-to-end delay:\\n');
260 fprintf('  E[T] = %.10f seconds\\n', T_end_to_end);
261
262 fprintf('\\nBottleneck:\\n');

```

```

263 fprintf(' current bottleneck = Q%d, rho = %.10f\n', bottleneck_current, max_rho);
264 fprintf(' lambda1_max = %.10f clients/sec\n', lambda1_max);
265 fprintf(' limiting queue for lambda1_max = Q%d\n', limiting_queue);
266
267 fprintf('\nDone.');
```

```

268
269 endfunction
270
271 % =====
272 % Required function for B.2
273 % =====
274
275 function [rho, ergodic] = intensities(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5,
    verbose)
276     % Computes traffic intensities rho_i for the five queues.
277     %
278     % Required public use:
279     % [rho, ergodic] = intensities(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5)
280     %
281     % Optional internal use:
282     % verbose = false suppresses printing.
283
284     if nargin < 8
285         verbose = true;
286     endif
287
288     mu = [mu1, mu2, mu3, mu4, mu5];
289     gamma = effective_arrival_rates(lambda1, lambda2);
290
291     rho = gamma ./ mu;
292     ergodic = all(rho < 1);
293
294     if verbose
295         fprintf('\nTraffic intensities rho_i = Gamma_i / mu_i:\n');
296         for i = 1:5
297             fprintf(' rho%d = %.10f\n', i, rho(i));
298         endfor
299     endif
300 endfunction
301
302 % =====
303 % Required function for B.3
304 % =====
305
306 function N = mean_clients(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5)
307     % Computes the mean number of clients in each M/M/1 queue:
308     %  $E[N_i] = \rho_i / (1 - \rho_i)$ 
309     %
310     % If the system is not ergodic, returns NaN values.
311
312     [rho, ergodic] = intensities(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5, false);
313
314     if ergodic == 0
315         N = NaN(1, 5);

```

```

316         return;
317     endif
318
319     N = rho ./ (1 - rho);
320 endfunction
321
322 % =====
323 % Helper functions
324 % =====
325
326 function gamma = effective_arrival_rates(lambda1, lambda2)
327     % Effective arrival rates at each queue, derived from the routing graph.
328
329     gamma1 = lambda1;
330     gamma2 = lambda2 + (1/2) * lambda1;
331     gamma3 = (5/12) * lambda1 + (1/3) * lambda2;
332     gamma4 = (1/3) * lambda1 + (2/3) * lambda2;
333     gamma5 = (37/60) * lambda1 + (11/15) * lambda2;
334
335     gamma = [gamma1, gamma2, gamma3, gamma4, gamma5];
336 endfunction
337
338 function T = end_to_end_delay(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5)
339     % Computes end-to-end mean delay using Little's Law:
340     %  $E[T] = E[N] / (\lambda_1 + \lambda_2)$ 
341
342     N = mean_clients(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5);
343     external_rate = lambda1 + lambda2;
344
345     if any(isnan(N)) || external_rate <= 0
346         T = NaN;
347         return;
348     endif
349
350     T = sum(N) / external_rate;
351 endfunction
352
353 function [lambda1_max, limiting_queue, bounds] = lambda1_max_for_ergodicity(lambda2,
mu1, mu2, mu3, mu4, mu5)
354     % Computes the maximum lambda1 such that all queues remain ergodic,
355     % with lambda2 and all mu_i fixed.
356     %
357     % Conditions:
358     % Gamma1 < mu1
359     % Gamma2 < mu2
360     % Gamma3 < mu3
361     % Gamma4 < mu4
362     % Gamma5 < mu5
363     %
364     % where:
365     % Gamma1 = lambda1
366     % Gamma2 = lambda2 + (1/2)lambda1
367     % Gamma3 = (5/12)lambda1 + (1/3)lambda2
368     % Gamma4 = (1/3)lambda1 + (2/3)lambda2

```

```
369 % Gamma5 = (37/60)lambda1 + (11/15)lambda2
370
371 bound1 = mu1;
372 bound2 = 2 * (mu2 - lambda2);
373 bound3 = (12/5) * (mu3 - (1/3) * lambda2);
374 bound4 = 3 * (mu4 - (2/3) * lambda2);
375 bound5 = (60/37) * (mu5 - (11/15) * lambda2);
376
377 bounds = [bound1, bound2, bound3, bound4, bound5];
378
379 [lambda1_max, limiting_queue] = min(bounds);
380
381 if lambda1_max < 0
382     lambda1_max = NaN;
383 endif
384 endfunction
385
```